硕士论文 / AI 编译器面试

从一段 matmul MLIR，到真实微内核执行

沿代码的变化讲论文：主线展示编译与执行，上方标出调度控制和研究落点。

A—D：论文研究 / 实现　　灰色：复用的编译基础设施　　---- 紫色虚线：调度控制 / 测量反馈　　主例：BLIS f32 6×16

A　调用知识建模

库源码 → Fact → Contract / Profile
硬条件：ISA / 布局 / tail / ABI
软偏好：分块与排序；保留来源和版本

B　Contract 驱动的候选空间

Task + Contract → 约束检查 / 活性分析 / 去重
direct / adapt / fallback / reject
输出：候选 token、分块参数、生成义务与诊断

BaCO 选择候选

复用搜索器
选中有效 token
按目标耗时搜索

D　联合验证与成本评价

数值 / IR / 符号 / 实际运行路径
目标计时回传；完整库与全成本对照
回答：调用是否成立，搜索是否值得

运行证据

Transform IR：描述“如何变换”的调度程序
bufferize → packing pass → 绑定微内核调用

选中 token → 填入参数
调度程序控制下方 IR 变换

程序编译与执行主链

1　Tensor matmul

```
%0 = linalg.matmul
  ins(%A, %B : ...)
  outs(%C : ...)
  -> tensor<?x?xf32>
```

输入：矩阵乘加的计算语义
尚未指定 packing 与微内核

2　缓冲区化

tensor → memref
linalg.matmul
操作内存中的 A/B/C
memref<?x?xf32>

one_shot_bufferize
复用 MLIR 缓冲区化

3　微内核接入 [C]

(a) packing / fringe pass
SCF 循环 + A/B panel packing
生成 direct / fringe 占位调用

(b) Transform op 绑定 callee
func.call @BLIS_wrapper(...)

产物：SCF / MemRef / Arith / func.call

4　标准 lowering

SCF → CF
MemRef / func / arith
→ LLVM dialect
↓ mlir-translate
LLVM IR (.ll)

复用标准转换与翻译工具

5　编译与链接

clang++ -O3
LLVM IR + driver
+ ABI adapter [C]
+ libblis.a
→ 可执行文件

adapter 对接 BLIS ABI

6　CPU 运行

driver 调用 GEMM
→ ABI adapter [C]
→ BLIS 6×16 微内核
输出 C + 执行耗时
记录真实调用与边界路径

微内核本体由 BLIS 提供

Fallback：通用 tiling / 可选 vectorization
先在 tensor 上调度，再 bufferization；接回标准 lowering

真实被调用的库内符号：
bli_sgemm_haswell_asm_6x16
尾块由生成代码处理；wrapper 对接 ABI

路径由 B 判定；reject 在候选阶段停止，不进入编译执行。

论文落点：在通用 lowering 前生成可调用的候选并物化微内核接入；在执行后验证路径、性能与搜索成本。

图中 IR 为节选示意。BLIS f64 复用核心生成逻辑；OpenBLAS RVV 保留包内生成与适配，不能套用此 BLIS pass。